



Tecnológico de Monterrey

Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 501)

Proyecto 3: Analizador Semántico

Sylvia Fernanda Colomo Fuente | A01781983

13 de mayo 2025

La semántica de C-	3
Tabla de símbolos	4
Reglas de inferencia.....	5
Uso de operadores aritméticos.....	5
Uso de operadores relacionales.....	5
Operación de asignación.....	5
Asignación de variables.....	5
Asignación de funciones.....	6
Tipos de return.....	6
Notas:.....	6

La semántica de C-

Este analizador semántico revisa la semántica dada para el lenguaje de C- algunos puntos claves a recordar son:

- Empieza de program a una declaration list
- Las variables se declaran al principio del código o función
- Solo se usan int
- Hay dos tipos int y void
 - El void se usará para arreglos o funciones únicamente
 - Una función de tipo void puede tener parámetros int pero debe tener una sentencia return de tipo void
- No hay funciones anidadas
- Existen los arrays de tipo int
- Las variables se declaran antes de usarse al igual que las funciones
- Las expresiones pueden ser aritméticas (+,-,/,*) relacionales (<, <=, >, >=, ==, !=) o de asignación (=)
- Las sentencias (if y while) evalúan expresiones de tipo int
- Los returns pueden retornar int o void

Tabla de símbolos

El diseño de la tabla de símbolos en este proyecto se basa en una estructura de clases orientadas a objetos. Cada *scope*, como el global o el cuerpo de una función, es representado mediante una instancia de la clase **SymbolTable**. Estas tablas se organizan jerárquicamente, permitiendo búsquedas ascendentes desde un *scope* local hacia sus *scopes* padre. Esto se logra dado que el analizador semántico mantiene un *scope* actual (*current_scope*) que se va actualizando conforme se recorren las funciones del programa. Al ingresar a una nueva función, se crea una nueva tabla hija (**SymbolTable**) y se cambia el *current_scope* a ella. Al terminar el recorrido del cuerpo de la función, el *current_scope* se restaura al anterior.

Cada tabla de símbolos va a contener un identificador del *scope* como puede ser global o el nombre de una función, el diccionario de símbolos, una lista de los *scopes* de bajo nivel (hijos), es decir las funciones del programa. Los hijos pueden llamar a su *scope* superior (parent) para buscar de manera recursiva sobre los símbolos declarados anteriormente. De este modo se puede presentar una tabla de símbolos la cual muestre el nombre, el tipo (*int*, *void*), los parámetros (si se habla de una función), si es un array (*bool*), si es un array muestra el tamaño (*size*) y finalmente la línea en donde se declara este símbolo.

Scope: global					
Nombre	Tipo	Parámetros	Es Array	Size	Línea
x	int	-	True	20	4
minimo	int	int [array], int	False	-	7
main	void	int	False	-	22
Scope: minimo					
Nombre	Tipo	Parámetros	Es Array	Size	Línea
a	int	-	True	void	7
high	int	-	False	-	7
i	int	-	False	-	8
x	int	-	False	-	8
k	int	-	False	-	8
Scope: main					
Nombre	Tipo	Parámetros	Es Array	Size	Línea
v	int	-	False	-	22
i	int	-	False	-	23
high	int	-	False	-	23

Reglas de inferencia

Uso de operadores aritméticos

+	-	*	/
$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 + e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 - e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 * e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 / e2: Int}$

Uso de operadores relacionales

<	<=	>	>=
$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 < e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 \leq e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 > e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 \geq e2: Int}$

!=	==
$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 \neq e2: Int}$	$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 == e2: Int}$

Operación de asignación

$$\frac{\vdash e1: Int, \vdash e2: Int}{\vdash e1 = e2: Int}$$

Asignación de variables

Simple	Array
$\frac{\vdash [a: Int] \vdash O(a): Int}{\vdash a: Int}$	$\frac{\vdash [a: Int[]] \vdash e: Int}{O \vdash a[e]: Int}$

Asignación de funciones

Int	Void
$\frac{O[f:Int] \vdash O(f):Int}{\vdash f:Int}$	$\frac{O[f:void] \vdash O(f):void}{\vdash f:void}$

Tipos de return

Int	Void
$\frac{O \vdash O(f):Int, e:Int}{\vdash f(e):Int}$	$\frac{O \vdash O(f):void}{\vdash f:void}$

Notas:

El analizador semántico también puede verificar que el índice de un arreglo esté dentro de los límites de la declaración y que el número de parámetros con los que se declaró una función sea igual al número con el que se llama. Sin embargo para estas verificaciones no hay regla de inferencia de TIPOS por lo que se establecen las reglas dentro de la función de **Typecheck**. También se revisa la existencia de una función principal main, la cual sino es registrada marca un error general y no se presentará la tabla de símbolos.